

Experiment-12 Full Stack Development

~Keshav, 23BIS70053

Code:

```
FULL STACK > EXP 12 > exp12 > ticket-booking > JS experiment4.3.js > ...
1  // experiment4.3.js
2
3  import express from 'express';
4  import { v4 as uuidv4 } from 'uuid';
5
6  const app = express();
7  app.use(express.json());
8
9  const seats = new Map();
10
11  // Initialize 10 seats with 'available' state
12  for (let i = 1; i <= 10; i++) {
13    | seats.set(String(i), { state: 'available' });
14  }
15
16  const LOCK_DURATION_MS = 60 * 1000; // 1 minute
17
18  function clearLock(seat) {
19    | if (seat.lockTimeoutId) {
20    |   | clearTimeout(seat.lockTimeoutId);
21    |   | seat.lockTimeoutId = undefined;
22    | }
23    | delete seat.lockId;
24    | delete seat.lockedAt;
25    | seat.state = 'available';
26  }
27
28  // GET all seats
29  app.get('/seats', (req, res) => {
30    | const result = {};
31    | for (const [id, seat] of seats.entries()) {
32    |   | result[id] = { state: seat.state };
33    | }
34    | res.json(result);
35  });|
36
37  // LOCK a seat
38  app.post('/lock/:id', (req, res) => {
39    | const id = String(req.params.id);
40    | const seat = seats.get(id);
41  });|
```

```

7 // LOCK a seat
8 app.post('/lock/:id', (req, res) => {
9   const id = String(req.params.id);
10  const seat = seats.get(id);
11
12  if (!seat) {
13    return res.status(404).json({ message: `Seat ${id} does not exist.` });
14  }
15
16  if (seat.state === 'booked') {
17    return res.status(400).json({ message: `Seat ${id} is already booked.` });
18  }
19
20  if (seat.state === 'locked') {
21    return res.status(400).json({ message: `Seat ${id} is already locked.` });
22  }
23
24  seat.state = 'locked';
25  seat.lockId = uuidv4();
26  seat.lockedAt = Date.now();
27
28  seat.lockTimeoutId = setTimeout(() => {
29    if (seat.state === 'locked') {
30      clearLock(seat);
31      console.log(`Auto-unlocked seat ${id} after timeout.`);
32    }
33  }, LOCK_DURATION_MS);
34
35  return res.status(200).json({
36    message: `Seat ${id} locked successfully. Confirm within 1 minute.`
37  });
38 });
39
40 // CONFIRM a seat
41 app.post('/confirm/:id', (req, res) => {
42   const id = String(req.params.id);
43   const seat = seats.get(id);

```

```
if (!seat) {
  return res.status(404).json({ message: `Seat ${id} does not exist.` });
}

if (seat.state !== 'locked') {
  return res.status(400).json({ message: 'Seat is not locked and cannot be booked.' });
}

if (seat.lockTimeoutId) {
  clearTimeout(seat.lockTimeoutId);
  seat.lockTimeoutId = undefined;
}

seat.state = 'booked';
delete seat.lockId;
delete seat.lockedAt;

return res.status(200).json({ message: `Seat ${id} booked successfully!` });
});

const PORT = 2000;
app.listen(PORT, () => {
  console.log(`Seat-locking server listening on http://localhost:\${PORT}`);
});
```

View in Postman Software:

GET http://localhost:2000/seats + ...

HTTP http://localhost:2000/seats Save

GET http://localhost:2000/seats Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Body Cookies Headers (7) Test Results 200 OK 50 ms 499 B Save Response

Pretty Raw Preview Visualize JSON

```
1  {
2    "1": {
3      "state": "available"
4    },
5    "2": {
6      "state": "available"
7    },
8    "3": {
9      "state": "available"
10   },
11   "4": {
12     "state": "available"
13   },
14   "5": {
15     "state": "available"
16   },
17   "6": {
18     "state": "available"
19   },
20   "7": {
21     "state": "available"
22   },
23   "8": {
```